4190.428 001 Introduction to Machine Learning

Generalization and Regularization
(Part2)

Jaesik Park
CSE & IPAI, SNU
jaesik.park@snu.ac.kr

Notes

- Please read Sec. 7 in our [BookS] textbook
  - [BookS] https://cs229.stanford.edu/main_notes.pdf
- This class will not address deep learning seriously, but the ideas theorems discussed in this class can be applied to deep learning algorithms.

Sample Complexity Bounds

ADVANCED

- We will (briefly) study the following theoretical topics:
  - Can we make formal the Bias-Variance Tradeoff?
  - Does doing well on the training set tell us anything about the test error?
  - Are there conditions under which we can actually prove that learning algorithms will work well?

- We are going to use these two lemmas:
  1. $P(A_1 \cup \cdots \cup A_k) \leq P(A_1) + \cdots + P(A_k)$
  2. Let $Z_1, \cdots, Z_n \sim^{i.i.d} Bernoulli(\phi)$ and $\hat{\phi} = \frac{1}{n}\sum Z_i$, and any $\gamma > 0$ be fixed then $P(|\phi - \hat{\phi}| > \gamma) \leq 2e^{-2\gamma^2 n}$ (Chernoff bound)

Sample Complexity Bounds

ADVANCED

- For simplicity, we will only consider a binary classification problem ($y \in \{0,1\}$)
  - but we can generalize this to other problems like regression
- We define **the training error** (empirical risk/error):

$$\hat{\epsilon}(h) = \frac{1}{n} \sum \mathbf{1}\{h(x^{(i)} \neq y^{(i)}\}$$

Indicator function

  - It is the fraction of training examples that h misclassifies
- We also define **the generalization error** $\epsilon(h) = P_{(x,y)\sim\mathscr{D}}(h(x) \neq y)$
  - Note that we assumed the training dataset and testing dataset share the same distribution
- **Hypothesis class** $\mathscr{H}$ is a set of all classifiers considered by it
  - For example, $\mathscr{H} = \{h_\theta \,|\, h_\theta(x) = 1\{\theta^\top x \geq 0\}, \theta \in \mathbb{R}^{d+1}\}$
  - We usually find $\hat{h} = \arg\min_{h \in \mathscr{H}} \hat{\epsilon}(h)$

Sample Complexity Bounds

ADVANCED

- Let $\mathscr{H} = \{h_1, \cdots, h_k\}$, and pick some h_i
- Using Chernoff bound, we can check $P(|\epsilon(h_i) - \hat{\epsilon}(h_i)| > \gamma) \leq 2e^{-2\gamma^2 n}$
- Letting $A_i :=$ event of $|\epsilon(h_i) - \hat{\epsilon}(h_i)| > \gamma$ leads to

$$P(\exists h \in \mathscr{H}, |\epsilon(h) - \hat{\epsilon}(h)| > \gamma) = P(A_1 \cup \cdots \cup A_k)$$

There exists

$$\leq \sum P(A_i) \leq \sum 2e^{-2\gamma^2 n} = 2ke^{-2\gamma^2 n}$$

- Subtracting 1 from both sides gives

$$P(\neg\exists h \in \mathscr{H}, |\epsilon(h) - \hat{\epsilon}(h)| > \gamma) = P(\forall h \in \mathscr{H}, |\epsilon(h) - \hat{\epsilon}(h)| \leq \gamma) \geq 1 - 2ke^{-2\gamma^2 n}$$

Negate

Sample Complexity Bounds

ADVANCED

- This gives the following **Theorem** and **Corollary** [BookS, Try yourself]:
- Let $|\mathscr{H}| = k$. Let $\delta > 0$ and γ be fixed. Then with probability at least $1 - \delta$, we have

$$\epsilon(\hat{h}) \leq \min_{h \in \mathscr{H}} \epsilon(h) + 2\sqrt{\frac{1}{2n} \log \frac{2k}{\delta}}$$

- **Corollary** : For $\epsilon(\hat{h}) \leq \min_{h \in \mathscr{H}} \epsilon(h) + 2\gamma$ to hold with probability at least $1 - \delta$, it suffices

$$n \geq \frac{1}{2\gamma^2} \log \frac{2k}{\delta} = O\left(\frac{1}{\gamma^2} \log \frac{k}{\delta}\right)$$

- We can check Bias-Variance Tradeoff:
  - since increasing $|\mathscr{H}| = k$ makes $\min_{h \in \mathscr{H}} \epsilon(h)$ smaller but $2\sqrt{\frac{1}{2n} \log \frac{2k}{\delta}}$ larger

Determining Number of Parameters

ADVANCED

- How can we really determine the number of parameters of a model?

- For example, there are $d + 1$ parameters at the linear classifier

$$h_\theta(x) = 1\{\theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d \geq 0\}$$

- But there are $2d + 2$ parameters at the same classifier

$$h_{u,v}(x) = 1\{(u_0^2 - v_0^2) + (u_1^2 - v_1^2)x_1 + \cdots + (u_d^2 - v_d^2)x_d \geq 0\}$$

- That's why we need the definition of Vapnik-Chervonenkis dimension!

Vapnik-Chervonenkis Dimension

ADVANCED

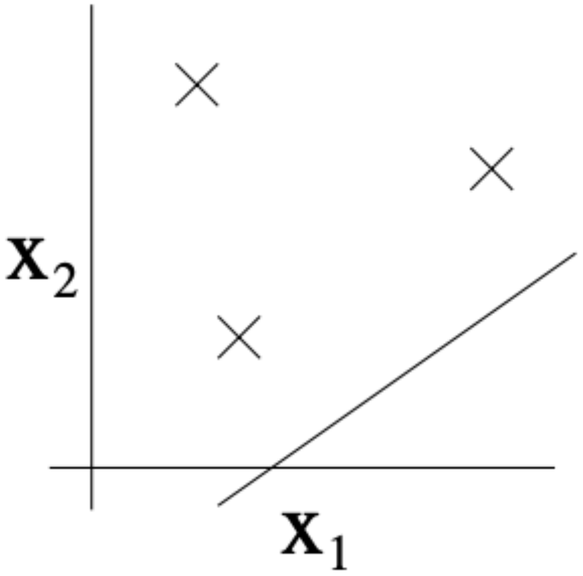
- Given a set $S = \{x^{(i)}, \cdots, x^{(D)}\}$, we say that $\mathscr{H}$ shatters S if for any set of labels $\{y^{(1)}, \cdots, y^{(D)}\}$, there exists some $h \in \mathscr{H}$ s.t. $h(x^{(i)}) = y^{(i)}$ for all $i = 1, \cdots, D$

- **Vapnik-Chervonenkis dimension** $VC(\mathscr{H})$ is the size of the largest set that is shattered by $\mathscr{H}$

Vapnik-Chervonenkis Dimension

ADVANCED

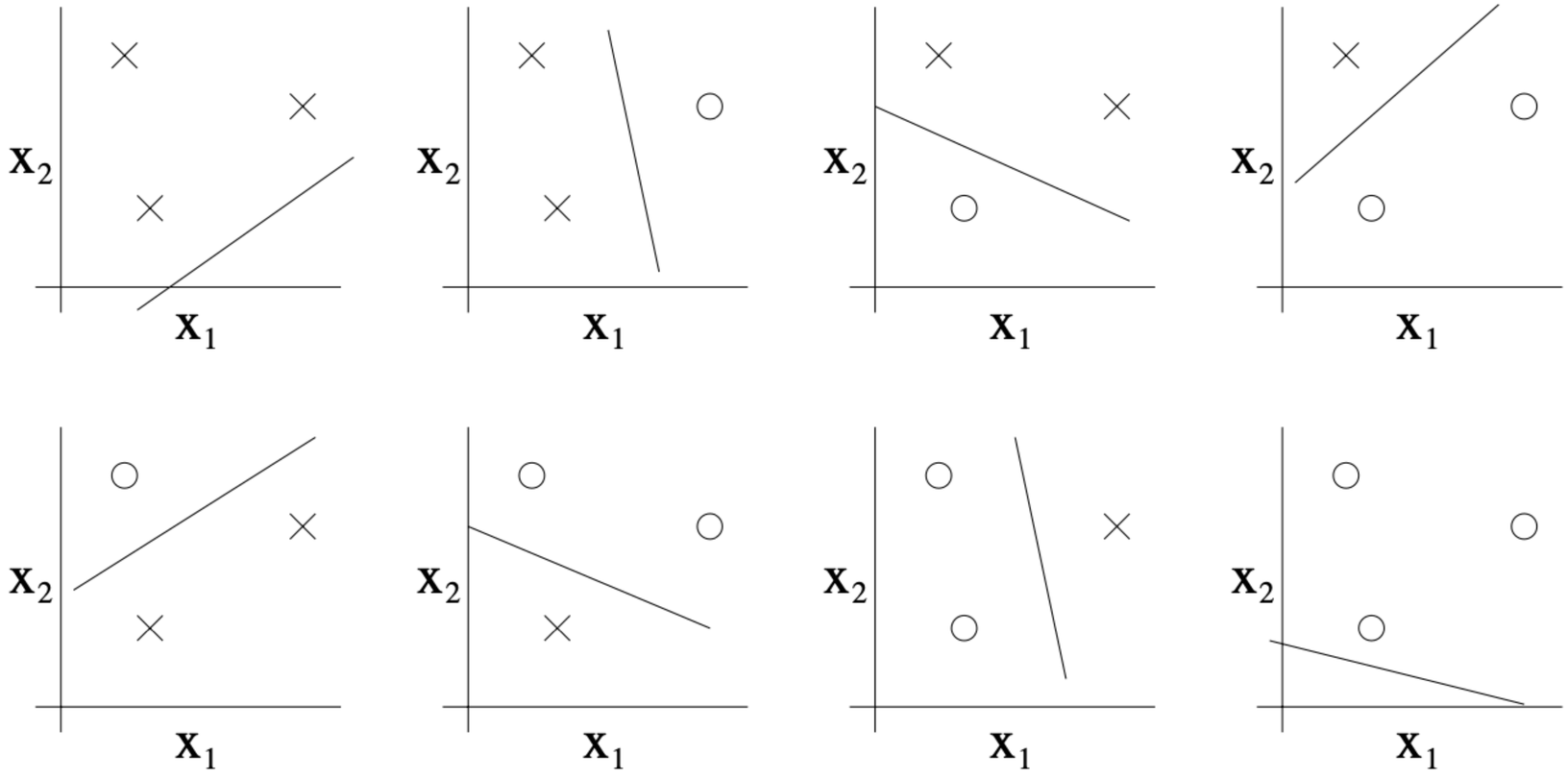
- For instance, consider the following set of three points
- Let $\mathscr{H} = \{h_\theta(x) \mid h_\theta(x) = 1\{\theta_0 + \theta_1 x_1 + \theta_2 x_2 \geq 0\}\}$, the set of 2-dimensional linear classifiers



Vapnik-Chervonenkis Dimension

ADVANCED

- As we can find a corresponding h for every eight possible labelings, and there is no set of 4 points that $\mathscr{H}$ can shatter, $VC(\mathscr{H}) = 3$



Theorem using VC Dimension

ADVANCED

- Let $\mathscr{H}$ be given, and let $D = VC(\mathscr{H})$. Then with probability at least $1 - \delta$, we have that for all $h \in \mathscr{H}$ [BookS, Try yourself],

$$|\epsilon(h) - \hat{\epsilon}(h)| \leq O\left(\sqrt{\frac{D}{n}\log\frac{n}{D} + \frac{1}{n}\log\frac{1}{\delta}}\right)$$

- That is, we have

$$\epsilon(\hat{h}) \leq \epsilon(h^*) + O\left(\sqrt{\frac{D}{n}\log\frac{n}{D} + \frac{1}{n}\log\frac{1}{\delta}}\right)$$

Theorem using VC Dimension

ADVANCED

- **Corollary** : For $|\epsilon(h) - \hat{\epsilon}(h)| \leq \gamma$ to hold for all $h \in \mathscr{H}$ with probability at least $1 - \delta$, it suffices that $n = O_{\gamma,\delta}(D)$. (Linear!)

- For most hypothesis classes, the VC dimension is (roughly) linear in the number of parameters. Thus...
- The number of training examples needed to achieve generalization error close to optimal error is usually **roughly linear** in the number of parameters of $\mathscr{H}$

Regularization

- Sometimes, the correct complexity measure of the models can be a function of the parameters (e.g. ℓ_2 norm of the parameters)
- In such cases, we will use **regularization**
- It controls the model complexity and prevent overfitting

Regularization

- $J_\lambda(\theta) = J(\theta) + \lambda R(\theta)$
  - $R(\theta)$ is called a **regularizer**
  - The Regularizer $R(\theta)$ is a **nonnegative** function
  - J_λ is often called **the regularized loss**
  - $\lambda \geq 0$ is called the **regularization parameter**
- We aim to find a model that fit the data and have a small model complexity!

Regularization Parameter and Regularizer

- The balance between the two objectives is controlled by λ
  - When $\lambda = 0$, the regularized loss is equivalent to the original loss
  - When λ is a small positive number, minimizing the original loss becomes the most important objective, with the regularizer as the tie-breaker
  - When λ is extremely large, then the original loss is not effective

Regularization Parameter and Regularizer

$$R(\theta) = \frac{1}{2}\|\theta\|_2^2$$

- **Option 1:** ℓ_2 **regularization** (in deep learning it is called **weight decay**)
- Most commonly used, compatible with **kernel trick**

$$R(\theta) = \|\theta\|_0$$

- **Option 2:** number of non-zeros in θ
- Imposing additional structure of the parameters narrows our search space, thus makes model simpler, and lead to a better generalization
- Since $\|\theta\|_0$ is not differentiable, a common relaxation is to use the ℓ_1 norm $R(\theta) = \|\theta\|_1$ (also called **LASSO**)

Weight Decay

- When applying gradient descent on the regularized loss $R_\lambda(\theta)$, we get: [BookS, Try yourself]

$$
\begin{aligned}
\theta &\leftarrow \theta - \eta \nabla J_\lambda(\theta) \\
&= \theta - \eta\lambda\theta - \eta \nabla J(\theta) \\
&= \underbrace{(1 - \eta\lambda)\theta}_{\text{decaying weights}} - \eta \nabla J(\theta)
\end{aligned}
$$

- It is equivalent to shrinking/decaying θ by a scalar factor of $1 - \eta\lambda$
- Hence it is referred to as **weight decay**

Implicit Regularization Effect

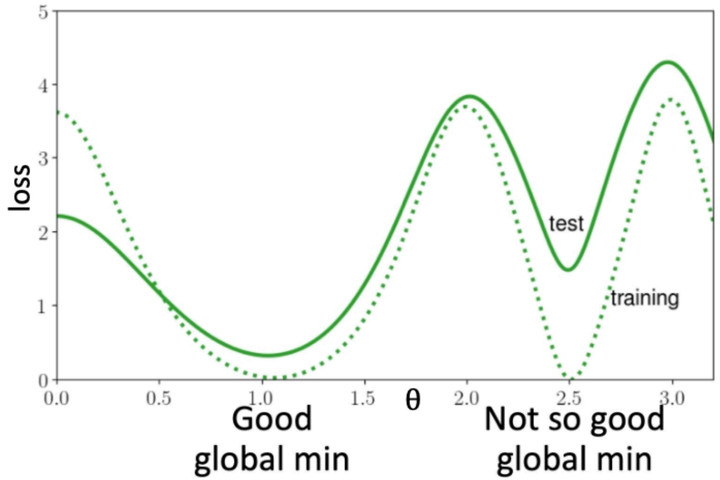
- Optimizers actually can implicitly impose structures on parameters beyond what has been imposed by the regularized loss

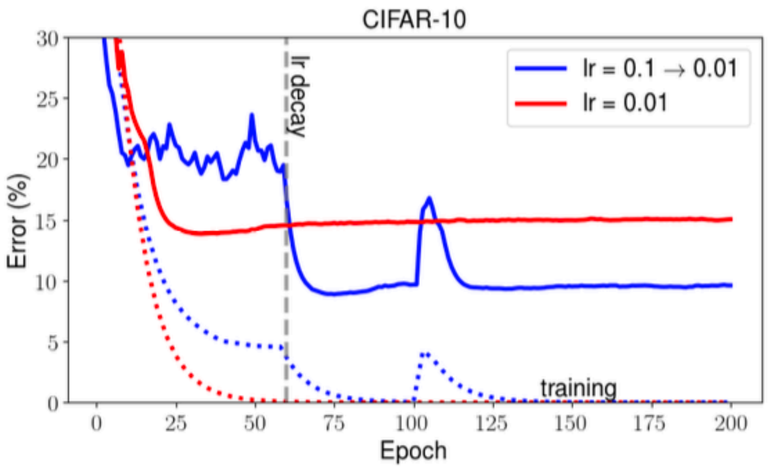
- **Classical Setting**: the loss has a unique global minimum
  - Any optimizer cannot impose any additional preferences

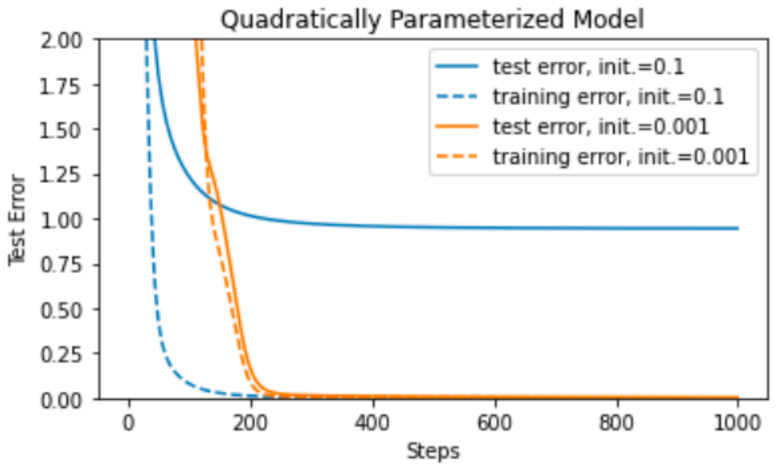
- **Deep Learning**: often the loss has more than one global minima
  - Difference optimizers may converge to different global minima
  - They can have dramatically different generalization performance even though they have the same training losses

Implicit Regularization Effect

- They can have dramatically different generalization performance even though they have the same training losses
- The choice of optimizer also imposes implicit regularization and affects the generalization of the model







Implicit Regularization Effect

- There is not the perfect answer, but we have some empirical results and heuristics
- In many situations, optimization can succeed in minimizing the training loss among those settings:
  - The use of larger initial learning rate
  - Smaller initialization
  - Smaller batch size
  - Momentum

Model selection via Cross Validation

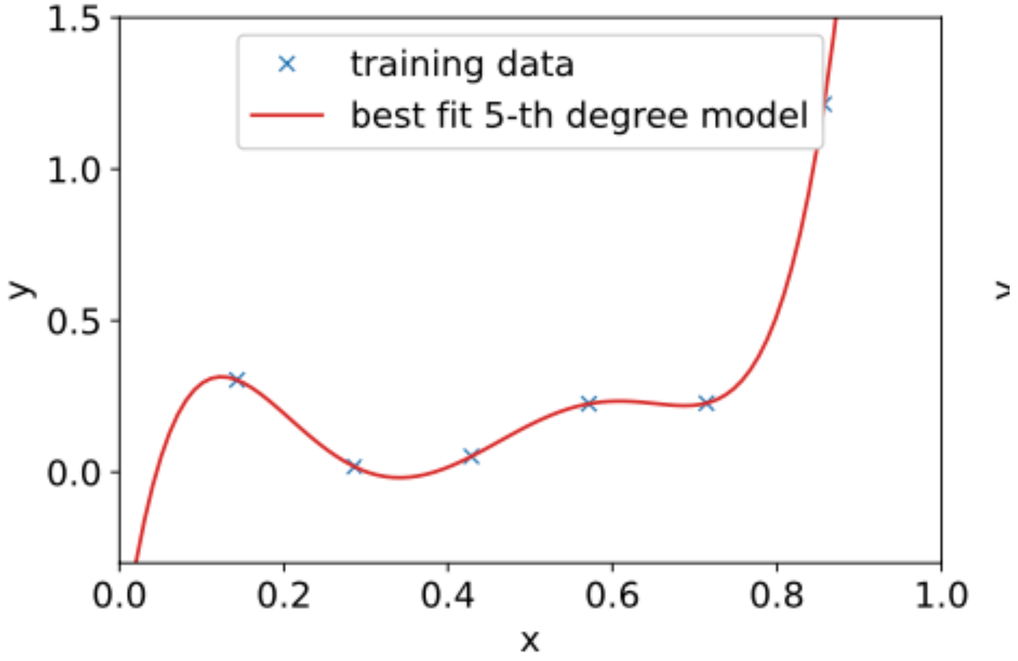
- For instance, we want to decide which k to use, at the polynomial regression model $h_\theta(x) = g(\theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_k x^k)$ $(k = 0,1,\cdots,10)$
- We may want to find the optimal parameter C for ℓ_1-regularized SVM
- How can we automatically select a model that represents a good tradeoff between the bias and variance?

Model selection via Cross Validation

- Naive approach: among the finite set of models $\mathscr{M} = \{M_1, \ldots, M_d\}$, let's use the following algorithm
- Given a training set S, train each model M_i on S to get some hypothesis h_i
- Then, pick the hypotheses with the smallest training error
- Is this work correctly? If not, what is the problem?

Model selection via Cross Validation

- It only concentrates on minimizing the training error
- It will always select a high-variance, high-degree polynomial model!



Hold-out(Simple) Cross Validation

1. Randomly split S into S_{train} and S_{CV}
   - We call S_{CV} the hold-out cross validation set
   - Practically, proportion of S_{CV} is about 30%
2. Train each model M_i on S_{train} only, to get some hypothesis h_i
3. Select and output the hypothesis h_i that had the smallest error $\hat{\varepsilon}_{S_{CV}}(h)$
   - $\hat{\varepsilon}_{S_{CV}}(h)$ denotes the average error of h on the set S_{CV}
   - The error on the set S_{CV} is also referred to as the **validation error**

Hold-out(Simple) Cross Validation

- We obtain a better estimate of each hypothesis h_i's true test error
  - We can pick the model with the smallest estimated test error
  - When the total dataset is huge, we can make fraction of S_{CV} smaller

- We can replace step 3 in the algorithm with :
  select the model $M_i = \arg\min_i \hat{\varepsilon}_{S_{CV}}(h_i)$ and retrain M_i on the entire S
  - However, it wastes about 30% of the whole data
  - It becomes a problem when the training data is scarce or expensive

k-Fold Cross Validation

1. Randomly split S into k disjoint subsets of m/k training examples each
   - Let these subsets be $S_1, \cdots, S_k$
2. For each model M_i, do the following:
   - For $j = 1, \cdots, k$, train M_i on $S \backslash S_j$ to get some hypothesis h_{ij}
   - After that, test h_{ij} on S_j to get $\hat{\varepsilon}_{S_j}(h_{ij})$
   - Then, calculate the estimated test error by averaging $\hat{\varepsilon}_{S_j}(h_{ij})$ over j
3. Pick the model M_i with the lowest estimated test error, and retrain that model on the entire S

k-Fold Cross Validation

- $k = 10$ is a typical choice for k-Fold CV
- When data is really scarce, we can use the extreme choice of $k = m$
  - We call this setting **Leave-one-out Cross Validation**
- Cross Validation is also can be used to evaluate a single model or algorithm

Frequentist vs Bayesian View

- **Frequentist View**
  - θ is unknown, constant-valued parameter
  - It is not random, we just don't know the exact value of it
  - Example : MLE (Maximum Likelihood Estimation)

$$\theta_{MLE} = \arg\max_{\theta} \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}; \theta)$$

Frequentist vs Bayesian View

- **Bayesian View**
  - θ is a random variable whose value is unknown
    - Our prior beliefs about the parameters: prior distribution $p(\theta)$
  - Example: Given a training set $S = \{(x^{(i)}, y^{(i)}\}_{i=1}^{n}$ and make a prediction on new x [Eq. Posterior]

$$p(\theta \mid S) = \frac{p(S \mid \theta) p(\theta)}{p(S)} = \frac{\left(\prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, \theta)\right) p(\theta)}{\int_{\theta} \left(\prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, \theta)\right) p(\theta) d\theta}$$

It is now a random variable, so we can use a comma instead

Bayesian Statistics

- We can find $p(y^{(i)} | x^{(i)}, \theta)$ from the model we are using at our problem

- Example :

$$p(y^{(i)} | x^{(i)}, \theta) = h_\theta(x^{(i)})^{y^{(i)}}(1 - h_\theta(x^{(i)}))^{(1-y^{(i)})}$$

at Bayesian logistic regression where

$$h_\theta(x^{(i)}) = \frac{1}{1 + exp(-\theta^\top x^{(i)})}$$

Bayesian Statistics

- When we are given a new test example x and make a prediction on it :
- We first compute our posterior distribution on the class label

$$p(y \mid x, S) = \int_{\theta} p(y \mid x, \theta) p(\theta \mid S) d\theta$$

- Then we may calculate the goal (for example, the expected value of $y \mid x$)

$$\mathbb{E}[y \mid x, S] = \int_{y} y p(y \mid x, S) dy$$

- This procedure is called **Fully Bayesian prediction**

Bayesian Statistics

- However, this Fully Bayesian prediction is computationally difficult in general
  - It requires taking integrals over the high-dim θ in [Eq. Posterior]
- We will instead approximate the posterior distribution for θ

Bayesian Statistics

- One common approximation: **MAP** (Maximum A Posteriori)

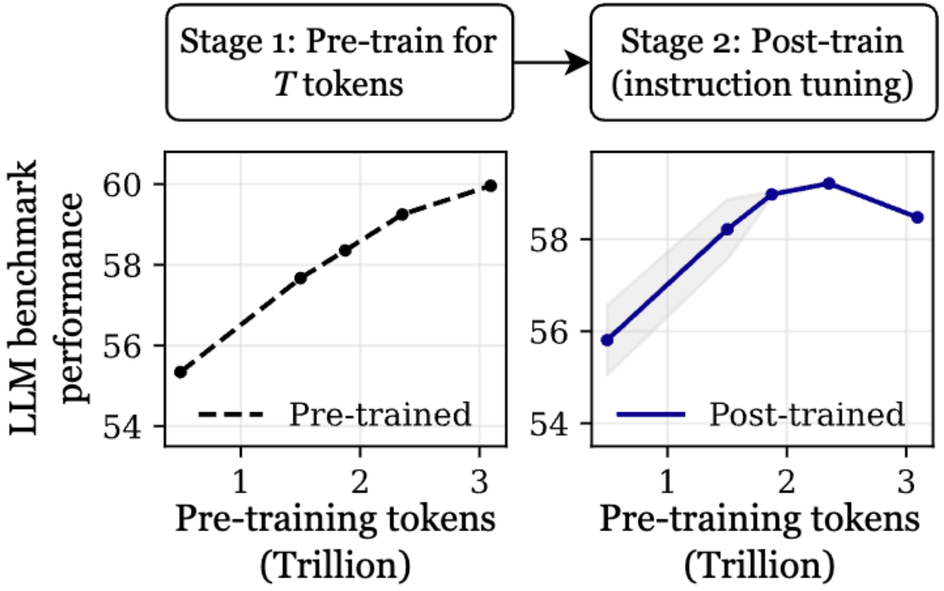
$$\theta_{MAP} = \arg\max_{\theta} \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, \theta) p(\theta)$$

Key difference from MLE

- Practically, common choice for the prior $p(\theta)$ is $\theta \sim \mathcal{N}(0, \tau^2 I)$
  - Using this choice, θ_{MAP} has smaller norm than θ_{MLE}
  - This causes the Bayesian MAP estimate to be less vulnerable to overfitting
- For example, Bayesian logistic regression at text classification ($d \gg n$)

Interesting paper

- Overtrained Language Models Are Harder to Fine-Tune (2025.03.28)
  - https://arxiv.org/abs/2503.19206
  - Extended pre-training can make models harder to fine-tune, leading to degraded final performance
  - Experiments on LLMs



QnA

- Any questions?
- Note: this lecture notes are largely adopted from
  - MIT 6.036(2019 Fall), Stanford CS229(2023-2024 Summer)